

Lab 2 - Data wrangling

Luke Owen

2026-03-16

```
library(tidyverse)
write_csv(midwest, "midwest.csv")
```

Questions

Part 1

Question 1

Calculate the number of counties in each state and display your results in descending order of number of counties. Which state has the highest number of counties, and how many? Which state has the lowest number, and how many?

```
midwest |>
  count(state, name = "n_counties") |>
  arrange(desc(n_counties))
```

```
# A tibble: 5 x 2
  state n_counties
  <chr>   <int>
1 IL      102
2 IN       92
3 OH       88
4 MI       83
5 WI       72
```

Illinois has the highest number of counties, with 102. Wisconsin has the lowest number of counties, with 72.

Question 2

A classmate says “Look at that, there is a county called ____ in each state in this dataset!” In a single pipeline, discover all counties that could fill in the blank. Your response should be a data frame with only the county names that could fill in the blank and how many times they appear in the data.

```
midwest |>
  count(county, name = "n") |>
  filter(n == 5)
```

```
# A tibble: 3 x 2
  county      n
  <chr>    <int>
1 CRAWFORD     5
2 JACKSON      5
3 MONROE       5
```

There are three counties that share names across all states. Crawford, Jackson, and Monroe.

Question 3

3a. The counties with a population density higher than 25,000. Your code must use the `filter()` function.

```
midwest |>
  select(county, state, popdensity, poptotal, area) |>
  filter(popdensity > 25000) |>
  arrange(desc(popdensity))
```

```
# A tibble: 9 x 5
  county      state popdensity poptotal  area
  <chr>      <chr>      <dbl>    <int> <dbl>
1 COOK       IL          88018.  5105067 0.058
2 MILWAUKEE WI          63952.   959275 0.015
3 WAYNE      MI          60334.  2111687 0.035
4 CUYAHOGA   OH          54313.  1412140 0.026
5 DU PAGE    IL          39083.   781666 0.02
6 MARION     IN          34659.   797159 0.023
7 HAMILTON   OH          34649.   866228 0.025
8 FRANKLIN   OH          28278.   961437 0.034
9 MACOMB     MI          25621.   717400 0.028
```

3b. The county with the highest population density. Your code must use the `max()` function.

```
midwest |>
  select(county, state, popdensity, poptotal, area) |>
  filter(popdensity == max(popdensity))
```

```
# A tibble: 1 x 5
  county state popdensity poptotal  area
  <chr>  <chr>      <dbl>    <int> <dbl>
1 COOK   IL          88018.  5105067 0.058
```

The county with the highest population density is Cook, IL.

Question 4

```
midwest |>
  summarize(
    median_density = median(popdensity),
    q1_density = quantile(popdensity, 0.25),
    q3_density = quantile(popdensity, 0.75)
  )
```

```
# A tibble: 1 x 3
  median_density q1_density q3_density
      <dbl>       <dbl>       <dbl>
1      1156.         622.        2330
```

The distribution of population density of counties is unimodal and extremely right-skewed. A typical Midwestern county has a population density of 1156 people per unit area. The middle 50% of counties have population densities between 622 and 2330 people per unit area.

Question 5

Another visualization from Lab 1 was the following that showed the proportion of urban counties in each state. Calculate the proportions in a single data pipeline.

```
midwest |>
  mutate(metro = if_else(inmetro == 1, "Yes", "No")) |>
  group_by(state) |>
  summarize(proportion_metro = mean(metro == "Yes"))
```

```
# A tibble: 5 x 2
  state proportion_metro
  <chr>           <dbl>
1 IL             0.275
2 IN             0.402
3 MI             0.301
4 OH             0.455
5 WI             0.278
```

Question 6

6.a. In a single pipeline, identify the observations marked in the orange square on the upper left corner. Your answer should be a data frame with four variables: county, state, percentage below poverty, and percentage college educated.

```
midwest |>
  filter(percollege < 10, percbelowpoverty > 30) |>
  select(county, state, percbelowpoverty, percollege)
```

```
# A tibble: 1 x 4
  county      state percbelowpoverty percollege
  <chr>      <chr>          <dbl>      <dbl>
1 MENOMINEE WI             48.7         7.34
```

6.b. In a single pipeline, identify the observations marked in the red square in the plot above. Your answer should again be a data frame with four variables: county, state, percentage below poverty, and percentage college educated.

```
midwest |>
  filter(percollege > 40, percbelowpoverty < 20) |>
  select(county, state, percbelowpoverty, percollege)
```

```
# A tibble: 5 x 4
  county      state percbelowpoverty percollege
  <chr>      <chr>          <dbl>      <dbl>
1 CHAMPAIGN IL             15.6         41.3
2 DU PAGE   IL              2.71         42.8
3 HAMILTON  IN              3.59         42.1
4 WASHTENAW MI             12.2         48.1
5 DANE      WI             10.5         43.6
```

6.c. Bring your answers from part (a) and part (b) together! In a single pipeline, and a single filter() statement, identify the observations marked in the red and orange square in the plot above. Your answer should again be a data frame with four variables: county, state, percentage below poverty, and percentage college educated.

```
midwest |>
  filter(
    (percollege < 10 & percbelowpoverty > 30) |
```

```
(percollege > 40 & percbelowpoverty < 20)
) |>
select(county, state, percbelowpoverty, percollege)
```

```
# A tibble: 6 x 4
  county    state percbelowpoverty percollege
  <chr>    <chr>          <dbl>      <dbl>
1 CHAMPAIGN IL          15.6        41.3
2 DU PAGE  IL           2.71        42.8
3 HAMILTON IN           3.59        42.1
4 WASHTENAW MI          12.2        48.1
5 DANE     WI           10.5        43.6
6 MENOMINEE WI          48.7         7.34
```

6.d. Create a new variable in midwest called `potential_outlier`. This variable should take on the value:

Yes if the point is one the ones you identified in part (c), i.e., one of the points marked in the squares in the plot above.

No otherwise.

Then, display the updated midwest data frame, with county, state, percentage below poverty, percentage college educated, `potential_outlier` as the selected variables, arranged in ascending order of `potential_outlier`.

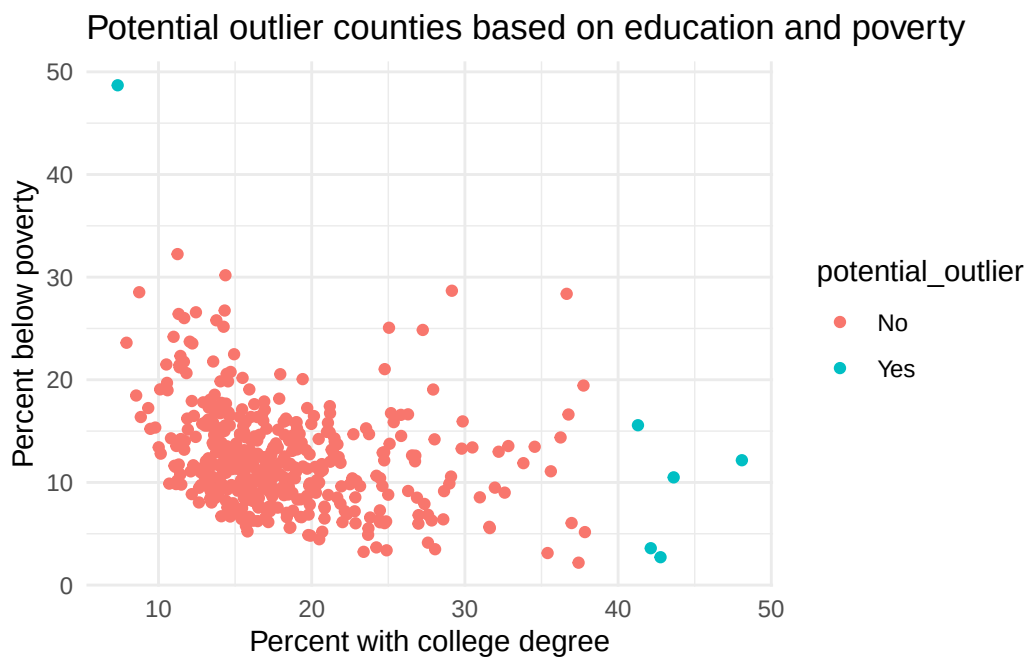
```
midwest <- midwest |>
  mutate(
    potential_outlier =
      if_else(
        (percollege < 10 & percbelowpoverty > 30) |
        (percollege > 40 & percbelowpoverty < 20),
        "Yes",
        "No"
      )
  )

midwest |>
  select(county, state, percbelowpoverty, percollege, potential_outlier) |>
  arrange(potential_outlier)
```

```
# A tibble: 437 x 5
  county      state percbelowpoverty percollege potential_outlier
  <chr>      <chr>          <dbl>         <dbl> <chr>
1 ADAMS      IL              13.2           19.6 No
2 ALEXANDER  IL              32.2           11.2 No
3 BOND       IL              12.1           17.0 No
4 BOONE      IL               7.21          17.3 No
5 BROWN      IL              13.5           14.5 No
6 BUREAU     IL              10.4           18.9 No
7 CALHOUN    IL              15.1           11.9 No
8 CARROLL    IL              11.7           16.2 No
9 CASS       IL              13.9           14.1 No
10 CHRISTIAN IL              11.7           13.6 No
# i 427 more rows
```

6.e. Recreate the visualization above, i.e. a scatterplot of percentage below poverty vs. percentage of people with a college degree, however color the points by the newly created potential_outlier variable instead of state.

```
ggplot(midwest, aes(x = percollege, y = percbelowpoverty, color = potential_outlier)) +
  geom_point() +
  labs(
    x = "Percent with college degree",
    y = "Percent below poverty",
    title = "Potential outlier counties based on education and poverty"
  ) +
  theme_minimal()
```

Question 7

7.a. In a single pipeline, calculate the total population for each state and save the resulting data frame as `state_population` and display it in descending order of total population.

```
state_population <- midwest |>
  group_by(state) |>
  summarize(total_population = sum(poptotal)) |>
  arrange(desc(total_population))

state_population
```

```
# A tibble: 5 x 2
  state total_population
  <chr>         <int>
1 IL           11430602
2 OH           10847115
3 MI           9295297
4 IN           5544159
5 WI           4891769
```

7.b. Then, in a separate pipeline, calculate the proportion of the total population in each state and, once again, display the results in descending order of proportion of population.

```
state_population |>
  mutate(population_proportion = total_population / sum(total_population)) |>
  arrange(desc(population_proportion))
```

```
# A tibble: 5 x 3
  state total_population population_proportion
  <chr>         <int>             <dbl>
1 IL           11430602             0.272
2 OH           10847115             0.258
3 MI           9295297              0.221
4 IN           5544159              0.132
5 WI           4891769              0.116
```

7.c. Which Midwestern state is most populous and what percent of the Midwest population lives there? Which is the least populous and what percent lives there?

The most populous Midwestern state is Illinois, and approximately 27.2% of the Midwest population lives there. The least populous state is Wisconsin, and approximately 11.6% of the Midwest population lives there.

Question 8

Question 8

Calculate the average percentage below poverty for each state and save the resulting data frame as `state_poverty` with the columns `state` and `mean_percbelowpoverty`.

Then, in a new pipeline, display the `state_poverty` data frame in ascending order of `mean_percbelowpoverty`. Which state has the lowest average percentage below poverty across its counties? Which state has the highest average percentage below poverty across its counties?

```
state_poverty <- midwest |>
  group_by(state) |>
  summarize(mean_percbelowpoverty = mean(percbelowpoverty))
state_poverty |>
  arrange(mean_percbelowpoverty)
```

```
# A tibble: 5 x 2
  state mean_percbelowpoverty
  <chr>           <dbl>
1 IN              10.3
2 WI              11.9
3 OH              13.0
4 IL              13.1
5 MI              14.2
```

Indiana has the lowest average percentage below poverty across its counties, at approximately 10.3%. Michigan has the highest average percentage below poverty, with an average of approximately 14.2% across its counties.

Part 2

Question 9

```
df <- tibble(  
  var_1 = c(10, 20, 30, 40, 50),  
  var_2 = c("Pizza", "Burger", "Pizza", "Pizza", "Burger"),  
  var_3 = c("Apple", "Apple", "Pear", "Pear", "Banana")  
)  
  
df
```

```
# A tibble: 5 x 3  
  var_1 var_2 var_3  
  <dbl> <chr> <chr>  
1     10 Pizza Apple  
2     20 Burger Apple  
3     30 Pizza Pear  
4     40 Pizza Pear  
5     50 Burger Banana
```

9.a. What does the following code chunk do? Run it and analyze the result and articulate in words what `arrange()` does.

```
df |>  
  arrange(var_2)
```

```
# A tibble: 5 x 3  
  var_1 var_2 var_3  
  <dbl> <chr> <chr>  
1     20 Burger Apple  
2     50 Burger Banana  
3     10 Pizza Apple  
4     30 Pizza Pear  
5     40 Pizza Pear
```

The `arrange()` function arranges the variable in ascending alphabetical order.

9.b. What does the following code chunk do? Run it and analyze the result and articulate in words what `group_by()` does. Also comment on how it's different from the `arrange()` in part (a)?

```
df |>
  group_by(var_2)
```

```
# A tibble: 5 x 3
# Groups:   var_2 [2]
  var_1 var_2 var_3
  <dbl> <chr> <chr>
1     10 Pizza Apple
2     20 Burger Apple
3     30 Pizza Pear
4     40 Pizza Pear
5     50 Burger Banana
```

`group_by(var_2)` groups the data by the values in `var_2`, so that future operations will be performed separately for each group. Unlike `arrange()`, it does not reorder the rows of the data frame; it simply adds grouping information to the dataset.

9.c. What does the following code chunk do? Run it and analyze the result and articulate in words what the pipeline does.

```
df |>
  group_by(var_2) |>
  summarize(mean_var_1 = mean(var_1))
```

```
# A tibble: 2 x 2
  var_2 mean_var_1
  <chr>     <dbl>
1 Burger      35
2 Pizza     26.7
```

This chunk of code first groups the data by `var_2` using `group_by()`. It then uses `summarize()` to calculate the mean value of `var_1` within each group. As a result, the output shows the average value of `var_1` for each category in `var_2`, producing one row per group.

9.d. What does the following code chunk do? Run it and analyze the result and articulate in words what the pipeline does. Then, comment on what the message says.

```
df |>
  group_by(var_2, var_3) |>
  summarize(mean_var_1 = mean(var_1))
```

```
`summarise()` has regrouped the output.
i Summaries were computed grouped by var_2 and var_3.
i Output is grouped by var_2.
i Use `summarise(.groups = "drop_last")` to silence this message.
i Use `summarise(.by = c(var_2, var_3))` for per-operation grouping
  (`?dplyr::dplyr_by`) instead.
```

```
# A tibble: 4 x 3
# Groups:   var_2 [2]
  var_2 var_3 mean_var_1
  <chr> <chr>      <dbl>
1 Burger Apple        20
2 Burger Banana       50
3 Pizza  Apple        10
4 Pizza  Pear         35
```

This pipeline groups the data by both `var_2` and `var_3`, creating groups based on each unique combination of those two variables. It then calculates the mean value of `var_1` for each group using `summarize()`.

The message printed by R indicates that the resulting data remains grouped by `var_2`, meaning that the grouping structure is partially retained after the summarization.

9.e. What does the following code chunk do? Run it and analyze the result and articulate in words what the pipeline does, especially mentioning what the `.groups` argument does. How is the output different from the one in part (d)?

```
df |>
  group_by(var_2, var_3) |>
  summarize(mean_var_1 = mean(var_1), .groups = "drop")
```

```
# A tibble: 4 x 3
  var_2 var_3 mean_var_1
  <chr> <chr>      <dbl>
1 Burger Apple        20
2 Burger Banana       50
3 Pizza  Apple        10
4 Pizza  Pear         35
```

This pipeline groups the data by both `var_2` and `var_3`, and then calculates the mean value of `var_1` for each group. The `.groups = "drop"` argument removes the grouping structure from

the output frame, returning back no remaining groups. This differs from part d, where the output remained grouped by `var_2`.

9.f. What do the following pipelines do? Run both and analyze their results and articulate in words what each pipeline does. How are the outputs of the two pipelines different?

```
df |>
  group_by(var_2, var_3) |>
  summarize(mean_var_1 = mean(var_1), .groups = "drop")
```

```
# A tibble: 4 x 3
  var_2 var_3 mean_var_1
  <chr> <chr>      <dbl>
1 Burger Apple       20
2 Burger Banana      50
3 Pizza  Apple       10
4 Pizza  Pear        35
```

```
df |>
  group_by(var_2, var_3) |>
  mutate(mean_var_1 = mean(var_1))
```

```
# A tibble: 5 x 4
# Groups:   var_2, var_3 [4]
  var_1 var_2 var_3 mean_var_1
  <dbl> <chr> <chr>      <dbl>
1    10 Pizza Apple       10
2    20 Burger Apple      20
3    30 Pizza Pear       35
4    40 Pizza Pear       35
5    50 Burger Banana     50
```

The first pipeline groups the data by `var_2` and `var_3` and uses `summarize()` to calculate the mean of `var_1` for each group, resulting in one row per group. The second pipeline also groups the data by `var_2` and `var_3`, but uses `mutate()` to add the group mean of `var_1` as a new column while keeping all original rows. Therefore, the outputs differ because `summarize()` reduces the number of rows while `mutate()` preserves the original dataset and adds additional information.